

Final Report: AI-Native Study Entry Submission

Final Report: AI-Native Study Entry Submission

1. Executive Summary

****Problem solved****: Developed a study entry submission application that validates input and detects duplicate titles before saving.

****Target user****: Students and educators who need a structured way to record study notes, homework progress, and research reflections.

****Value proposition****: Unlike a generic form, this project delivers rigorous data validation, clean architecture, and an accessible UI governed by a strict design contract.

2. Engineering Harness

AGENTS.md

The repository includes ``AGENTS.md``, which defines constraints for AI agents and the development process:

- Only validated architectural patterns are allowed.
- All decisions must be documented in ``/docs``.
- Unstructured "vibe code" is prohibited.
- UI components must be accessible and predictable.

BDD and requirements

The feature acceptance criteria are defined in ``docs/requirements/feature_study_entry_submission.md`` using a BDD-style approach:

- Title is required.
- Content is required.
- Duplicate titles are rejected.

These rules ensure the application logic remains deterministic and testable.

3. Architecture & Logic

Pattern selection

The business logic uses the ****Strategy**** pattern. In ``src/study_entry_submission/strategy.py`` the following classes are implemented:

- ``SubmissionStrategy`` - base strategy interface.
- ``ValidSubmissionStrategy`` - validates correct input and normalizes the entry.
- ``InvalidSubmissionStrategy`` - collects validation errors.
- ``SubmissionContext`` - delegates to the appropriate strategy.

Architecture diagrams

Final Report: AI-Native Study Entry Submission

The interaction flow is documented in ``docs/architecture/flow_study_entry_submission.md``:

- UI -> Application Service -> Strategy -> Pure Function
- Response cases: ``accepted``, ``validation_failed``, ``duplicate_entry``

Pure function logic

The function ``evaluate_entry_submission(input_data, existing_titles, date)`` is pure and side-effect free. It returns a single deterministic response object.

4. UI & Integration

DESIGN.md

The UI contract is defined in ``docs/DESIGN.md`` and includes:

- Framework: Streamlit
- Color palette: ``#1F2937``, ``#3B82F6``, ``#F8FAFC``, ``#FFFFFF``, ``#E2E8F0``, ``#16A34A``, ``#F59E0B``, ``#DC2626``
- Typography and spacing rules
- Component rules for form labels, button states, feedback panels, and accessibility

UI implementation

The frontend is implemented in ``src/study_entry_submission/app.py`` and includes:

- a title input field
- a content text area
- a date picker
- a submit button
- a response section showing the backend payload

Integration

The UI directly calls ``evaluate_entry_submission(...)``. On valid submission, the app shows an accepted response; on invalid or duplicate input, it shows field-level validation messages or a duplicate warning.

5. Deployment Configuration

Ready for deployment

The files required for deployment are:

- ``requirements.txt``
- ``streamlit_app.py``
- ``docs/DEPLOYMENT.md``

Hosting platform

Final Report: AI-Native Study Entry Submission

Recommended hosting platform: **Streamlit Community Cloud**.

Run command

```
`streamlit run streamlit_app.py`
```

6. PKM Insights & Retrospective

Experiment logs

- `experiments/EXP-2026-05-16-pattern-implementation.md` - Strategy pattern implementation
- `experiments/EXP-2026-05-16-pure-function.md` - pure function design
- `experiments/EXP-2026-06-13-spec-driven-ui.md` - spec-driven UI experiment

Key lessons

- **Challenge**: The main challenge was keeping the AI constrained within a strict architectural and accessibility contract.
- **Overengineering**: It is important not to convert a simple user input flow into unnecessary architectural complexity.
- **Hallucinations**: The AI tends to generate generic or vague code unless given a strong design contract.
- **Engineer role**: The software engineer becomes an architect and verifier, not just an implementer; quality control, documentation, and validation are essential.

7. File list

- `docs/DESIGN.md`
- `docs/DEPLOYMENT.md`
- `docs/architecture/flow\_study\_entry\_submission.md`
- `docs/requirements/feature\_study\_entry\_submission.md`
- `src/study\_entry\_submission/strategy.py`
- `src/study\_entry\_submission/app.py`
- `src/study\_entry\_submission/README.md`
- `streamlit\_app.py`
- `requirements.txt`
- `experiments/EXP-2026-06-13-spec-driven-ui.md`